

CSA17 - Scenario Based Assignment Report

Artificial Intelligence - Assessment Tool 2 (CO2)

Name: Roshitha

Reg No: 192424400

Q1. AI Drone Delivery Routing (Greedy vs A*)

Problem Explanation

A drone must route through a flood-affected region with a heuristic estimate, partial real-time blocked-path updates, and variable movement costs, choosing between Greedy Best-First Search and A*.

Approach

Greedy Best-First Search expands purely by heuristic $h(n)$; A* expands by $f(n) = g(n) + h(n)$, balancing accumulated cost with the heuristic estimate. A dynamic edge block is also simulated to test re-planning.

Python Code

```
graph = {
    "Depot": [("A", 2), ("B", 5)], "A": [("C", 2), ("E", 9)],
    "B": [("D", 2)], "C": [("FloodZone", 2)],
    "D": [("E", 2)], "E": [("FloodZone", 2)], "FloodZone": [],
}
HEURISTIC = {"Depot": 8, "A": 6, "B": 9, "C": 7, "D": 5, "E": 1, "FloodZone": 0}

def greedy_best_first(start, goal, blocked=set()):
    frontier = [(h(start), start)]
    ...
    # expands purely by h(n)

def astar(start, goal, blocked=set()):
    frontier = [(h(start), 0, start)]
    ...
    # expands by f(n) = g(n) + h(n)
```

Output

Greedy path: Depot→A→E→FloodZone, cost 13 (misled by an inflated heuristic at node C).
A* path: Depot→A→C→FloodZone, cost 6 (optimal). After the C→FloodZone edge is blocked, A* re-plans to Depot→A→E→FloodZone, cost 13.

Q2. Traffic Signal Optimization (Hill Climbing vs Simulated Annealing)

Problem Explanation

A smart city must tune signal timings to minimize waiting time, fuel use, and congestion over a huge, dynamically changing search space prone to local optima.

Approach

Hill Climbing only accepts improving neighbour moves; Simulated Annealing also accepts some worse moves (with probability that shrinks as temperature cools) to escape local optima.

Python Code

```
def waiting_time_cost(signal_timing):
    x = signal_timing
    return 0.002*(x-70)**2 + 8*math.sin(x/6)**2 + 0.5

def hill_climbing(start, step=1, max_iter=200):
    current = start
    for _ in range(max_iter):
        neighbours = [current-step, current+step]
        best = min(neighbours, key=waiting_time_cost)
        if waiting_time_cost(best) >= waiting_time_cost(current):
            break # stuck at local optimum
        current = best
    return current, waiting_time_cost(current)

def simulated_annealing(start, max_iter=5000, T0=20.0, cooling=0.998):
    current = start
    T = T0
    for _ in range(max_iter):
        candidate = current + random.choice([-1,1])*random.randint(1,6)
        delta = waiting_time_cost(candidate) - waiting_time_cost(current)
        if delta < 0 or random.random() < math.exp(-delta/T):
            current = candidate
        T *= cooling
    return current, waiting_time_cost(current)
```

Output

Hill Climbing starting at 20 got stuck at 19 (cost 5.707). Simulated Annealing starting at the same point converged near 75 (cost 0.585) — a far better solution, demonstrating its ability to escape the local optimum.

Q3. Mars Rover Online Search Agent

Problem Explanation

A rover explores unmapped Martian terrain with limited sensing range and must reach a sample-collection goal while avoiding hazards, updating its knowledge as it goes.

Approach

An online search agent repeats sense → plan (Uniform Cost Search over currently known cells) → act, one step at a time, re-planning whenever a new hazard is sensed.

Python Code

```
def online_rover_search(start, goal, terrain):
    known = {start: terrain[start[0]][start[1]]}
    current, travelled = start, [start]
    while current != goal:
        known.update(sense(current, terrain))
```

```

plan = ucs_plan(current, goal, known) # UCS over known cells only
current = plan[1]
travelled.append(current)
return travelled

```

Output

Final path travelled: (0,0)→(0,1)→(0,2)→(1,2)→(2,2)→(2,3)→(2,4)→(3,4)→(4,4), with the route re-planned at every step as new hazards were sensed.

Q4. Exam Timetable Scheduling (CSP)

Problem Explanation

A university must assign exam slots to courses so that no student has overlapping exams and other resourcing constraints are respected, at a scale that makes brute-force checking infeasible.

Approach

Backtracking Search with Minimum-Remaining-Values (MRV) ordering and Forward Checking: MRV always schedules the most-constrained course next, and forward checking removes newly-invalid slots from other courses' domains immediately after each assignment.

Python Code

```

def forward_check_exam(domains, assigned_course, assigned_slot):
    new_domains = {c: set(v) for c, v in domains.items()}
    for c in COURSES:
        if c != assigned_course and conflicts(c, assigned_course):
            new_domains[c].discard(assigned_slot)
    return new_domains

def select_mrv(domains, assignment):
    unassigned = [c for c in COURSES if c not in assignment]
    return min(unassigned, key=lambda c: len(domains[c]))

def backtrack_exam(assignment, domains):
    if len(assignment) == len(COURSES):
        return dict(assignment)
    course = select_mrv(domains, assignment)
    for slot in sorted(domains[course]):
        assignment[course] = slot
        pruned = forward_check_exam(domains, course, slot)
        if all(len(pruned[c]) > 0 for c in COURSES if c not in assignment):
            result = backtrack_exam(assignment, pruned)
            if result is not None:
                return result
        del assignment[course]
    return None

```

Output

Final exam timetable: AI = Slot1, DBMS = Slot2, OS = Slot1, CN = Slot2, ML = Slot3, found with zero backtracking thanks to MRV + forward checking.

Q5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

Problem Explanation

A real-time strategy game AI must choose moves within strict time limits despite an extremely large game tree, requiring an efficient way to search for the best move.

Approach

Minimax evaluates the full game tree by alternating maximizing/minimizing players; Alpha-Beta pruning adds alpha/beta bounds to cut off branches that cannot affect the final decision, reaching the same result while visiting far fewer nodes.

Python Code

```
def minimax(tree, depth, maximizing):
    if depth == 0:
        return tree
    if maximizing:
        return max(minimax(c, depth-1, False) for c in tree)
    return min(minimax(c, depth-1, True) for c in tree)

def alphabeta(tree, depth, alpha, beta, maximizing):
    if depth == 0:
        return tree
    if maximizing:
        value = float("-inf")
        for child in tree:
            value = max(value, alphabeta(child, depth-1, alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                break # beta cut-off
        return value
    else:
        value = float("inf")
        for child in tree:
            value = min(value, alphabeta(child, depth-1, alpha, beta, True))
            beta = min(beta, value)
            if alpha >= beta:
                break # alpha cut-off
        return value
```

Output

Both Minimax and Alpha-Beta Pruning return the same optimal value (5) on the sample game tree, but plain Minimax visits 15 nodes while Alpha-Beta visits only 11 — a saving of 4 nodes on this small tree, with much larger savings expected on deeper, real game trees.